Chapter 6. Improving Reliability

Foundational models are inherently *stochastic*, which means they involve random variables or probability. As a result, GenAI applications built on top of these models can suffer from inconsistent output, factual inaccuracies, and hallucinations. This chapter introduces four patterns that are designed to mitigate these challenges.

LLM-as-Judge (Pattern 17) facilitates systematic evaluation of output by employing LLMs, either directly via prompting or through ML/fine-tuned models, to assess quality. Reflection (Pattern 18) enables models to critique their own work and iteratively refine generated content. Dependency Injection (Pattern 19) promotes testability and robustness by enabling the mocking of components. Finally, Prompt Optimization (Pattern 20) offers a structured approach to refining input prompts and thus maximizes output reliability across diverse input distributions.

Together, these patterns form a framework that enhances the dependability of GenAI applications. By integrating mechanisms for evaluation, self-correction, uncertainty quantification, modular design, and input optimization, you can build more reliable systems and foster greater user confidence in the outputs generated

5. Extending Model Capabilities

6. Improving Reliability
Generative AI Design Patterns

7. Enabling Agents to Take Action

The LLM-as-Judge pattern is a way to provide detailed, multidimensional feedback that you can use to compare models, track improvements, and guide further development. LLM-as-Judge represents a promising middle ground between fully automated metrics and human evaluation, and it offers scalable yet nuanced assessment capabilities for GenAI systems.

Problem

Many of the patterns discussed in this book incorporate an evaluation step. In Content Optimization (Pattern 5), for example, you compare two pieces of content. In Node Postprocessing (Pattern 10), you rerank retrieved chunks on relevance. In Tree of Thought (Pattern 14), you evaluate paths and choose the most promising one. In Evol-Instruct (Pattern 16), you evaluate examples and prune any that would corrupt the dataset.

Effective evaluation is fundamental to using GenAI effectively. It provides feedback loops that drive model improvements and refinements. Without robust evaluation, it's difficult to determine whether AI outputs meet your standards or requirements. Reliable evaluation methods build users' confidence in AI systems by demonstrating those systems' capabilities and limitations. Proper evaluation can help you identify potential biases, inaccuracies, or harmful outputs before you deploy your model.

Evaluating GenAI systems' capabilities is often hard, however, because the tasks they perform are open-ended. It's relatively straightforward to evaluate the performance of a mousetrap—you simply count how many mice it catches—but it's harder to evaluate marketing content because the following traditional evaluation approaches have several limitations:

Outcome measurement

> The gold standard of evaluation in the enterprise is to measure outcomes. Ideally, you'd be able to tell how good a marketing brochure is by measuring the increase in sales that it drives. However, sales increases can be attributed to a host of factors. Because no two customers or days are alike, it's hard to say how much of any change in sales volume was sparked by a single piece of marketing content. This is why we use *outcome proxies*, such as engagement—proxies let us attribute clicks to specific pieces of content, and we can use approaches such as multiarmed bandits to reduce testing times.

Human evaluation

> The next-best option is to have human experts score the marketing content.

The benefit of this approach is that it can be done without actually deploying the content. Unfortunately, human evaluation is difficult to scale—it's expensive, time-consuming, subject to individual biases, and limited by the experts' availability.

Automated metrics

The NLP literature suggests the use of metrics such as BLEU and ROUGE to measure content's fitness for translation and summarization, respectively. The advantage of these metrics is that they're scores that can be calculated in a systematic way. They work by finding the n-grams that a generated translation or summary and a set of reference translations or summaries have in common. However, these metrics fail to capture semantic meaning, nuance, and factual correctness. They also fail to be relevant to the actual outcome that we're seeking to drive—you can't customize a BLEU score to make it more relevant to the problem of creating marketing copy to drive heavy-machinery sales.

For this pattern, we'll set out to come up with a way to evaluate GenAI responses that combines the benefits of these three approaches—meaning it should be a good proxy for outcomes, be usable without actually deploying the model, and be calculated systematically—but is also fast, scalable, and customizable.

Solution

LLM-as-Judge leverages the capabilities of advanced language models to evaluate outputs. This approach scales more effectively than human evaluation, provides more nuanced assessment than traditional automated metrics, can be customized for specific evaluation criteria, and applies evaluation standards consistently.

There are three ways to apply LLM-as-Judge. Option 1, shown in the left side of Figure 6-1, is to employ a prompting approach. Option 2 is to use ML, and option 3 is to use a fine-tuned LLM. Both the ML and fine-tuning approaches incorporate prompting—very often, you'll start with the prompting approach and then improve LLM-as-Judge by either ML or fine-tuning. In the ML approach, you send the output of the prompting approach to an ML model. In the fine-tuned approach, you replace the LLM used in prompting with Adapter Tuning (Pattern 15). Let's look at these approaches one by one.

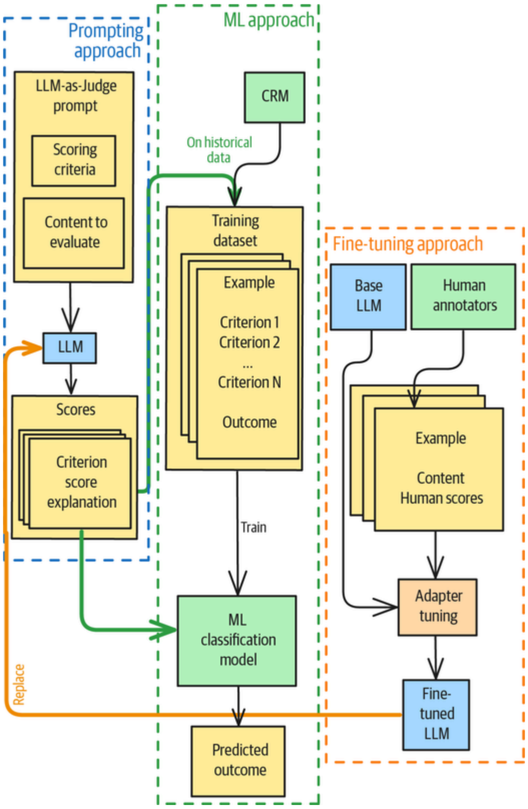


Figure 6-1. Three approaches to LLM-as-Judge: Prompting, ML, and fine-tuning

Option 1: Prompting approach

LLM-as-Judge involves creating a custom scoring rubric for your problem and then asking an LLM to apply the rubric to the given output. For example, you could use this prompt to ask an LLM to evaluate a summary:

> Given an article and a summary, provide a score in the range of 1-5 for each of the following criteria:
>
> - Factual accuracy
>
> - Completeness of key points
>
> - Conciseness
>
> - Clarity
>
> For each score, provide a brief justification.
>
> **Article**
>
> {article}
>
> **Summary**
>
> {summary}
>
> **Scores**:

You have to guard against the same prompt returning different scores on the same inputs on different invocations, so make sure to set the temperature to zero. Doing client-side caching or server-side prompt/context caching can help when it's absolutely essential that scores are repeatable.

An important step that helps with consistency is to preprocess the input data to make it more self-contained. For example, when evaluating conversational answers, it may be important to include enough turns of the conversation and to summarize the agent's answers.

Another practice that helps with consistency is to expand on the calibration rubric. For example, instead of just saying in your prompt that you want factual accuracy, you can explain what factual accuracy means to you:

> - Factual accuracy
>
>   * 1 if any information in the summary misrepresents the article
>   * 5 if all statements in the summary are grounded in the article
>
> - Completeness of key points
>
>   * 1 if multiple high-impact points are missing
>   * 3 if all the major points are present
>   * 5 if all the major points are present and the more important points receive more coverage in the summary
>
> - Conciseness
>
>   * 1 if the summary contains redundant information
>   * 5 if the summary efficiently conveys information without unnecessary details
>
> - Clarity
>
>   * 1 if there is awkward phrasing or technical jargon used without explanation
>   * 5 if the summary is well structured and is easy for a 10th grader to understand

This example shows a numerical scoring scale, but that's not the only way to evaluate the response. There are some key questions to consider when choosing an evaluation approach:

> *What are you evaluating?*
>
> > Do you need to score a single piece of content, or do you need the LLM to identify which of two potential choices is better? Sometimes, both choices

are generated content but one is a reference answer (perhaps produced by the current, non-LLM system). Or instead of just two choices, you might have a list of content pieces that need to be ranked, with one choice being a reference answer.

How will you use the scores?

Do you want the evaluation result to be a binary value, a numerical score, a ranking, or a categorical value? Does the metric need to align with true outcomes or with human preferences? This comes down to how you will use the scores. For example, if you plan to make a decision based on the output of the model, a binary value is best because it will let you clearly tie the score to the correctness of the decision.

How will humans be involved?

If humans will use the scores, you typically need human interpretability, so you should ask the LLM to generate explanations along with the scores. If humans are involved during the evaluation, you could make the evaluation multistage and provide feedback to improve the evaluation. It could even be a conversational "cross-examination" in which some (or all) feedback is provided by an LLM.

Modify your scoring rubric based on these considerations.

Option 2: ML

Ideally, the score generated by the LLM-as-Judge pattern will be consistent across the content you want to drive and will indicate the outcome you want. You can achieve this by creating an ML model to combine the scores LLM-as-Judge outputs into a single score that predicts the outcome. To evaluate marketing content by using this approach, you'll follow these three steps:

Step 1: Create a scoring rubric for prompting

Establish clear criteria for what makes marketing copy effective at driving sales. Perhaps your scoring rubric includes criteria such as how clear the call to action is, how well the value proposition is communicated, how well it speaks to the intended customer profile, and how well it describes the product's uniqueness and differentiates it from competitors' products.

Step 2: Collect historical data

Collect a diverse dataset of marketing copy for which historical outcome data is available. If possible, look to your customer relationship management (CRM) software. For example, you might use your CRM software to identify cases when a salesperson sent a marketing brochure to a customer and then note whether or not the customer purchased the product in question within, say, 30 days of receiving the brochure.

Step 3: Train a classification model

Score the marketing brochures by using the LLM-as-Judge prompting approach and assign each score an outcome, noting whether or not a purchase was made. This gives you a training dataset consisting of the scoring-rubric values and eventual outcomes, which you can use to train an ML classification model.

One advantage of this approach is that, because you are training on LLM-produced scoring rubrics and real outcomes, the ML model will discount any criteria that don't matter or on which the LLM is inconsistent (as long as you don't overfit,[1] of course).

Often, you'll start with the prompting method and then migrate to the ML approach once you have measured some outcomes and can train a model to produce scores that "predict" the desired outcome.

Option 3: Fine-tuning

Writing calibration criteria for LLM-as-Judge can be hard. What makes marketing content "persuasive," for example? In such cases, it can help to have human experts annotate and score content with the same scoring rubric as the LLM-as-Judge. This gives you a set of ideal input-response pairs, and you can use them to carry out Adapter Tuning (see Chapter 5) to create a fine-tuned model that's capable of producing scores similar to those produced by human experts.

A fine-tuning approach like this works in situations like applying medical diagnostics checklists to patient records when you want the LLM to mimic how humans would do it—for example, in a way that conforms with medical standards and how medical experts would interpret the observations.

Example

Suppose you want to evaluate the quality of text in a voters' guide, like this argument from a US one for a 2022 Initiative in Washington State:

> Washington State has taken important steps to keep guns out of dangerous hands. But there are still gaps in our laws that make it hard to keep guns away from people threatening violence against themselves or others. We know that the majority of mass shooters and individuals who attempt suicide show signs of their intentions, but current law leaves families and law enforcement—often first to see those warning signs—unable to take life-saving action.
>
> ...

How good is the argument against the same initiative?

> I-1491 disregards existing state laws that already require treatment and restriction of potentially dangerous individuals. I-1491 doesn't require evaluation, treatment, or monitoring and does nothing to address underlying issues. Recently implemented laws actually provide early detection and intervention of persons at danger to themselves or others.
>
> ...

Let's say that you want the arguments to be persuasive enough to cause more people to vote for or against the initiative, respectively. How do you evaluate the arguments against these goals?

Choosing an evaluation method

The following traditional approaches fail in this scenario:

> *Outcome measurement*
>
> > The outcome is the election result, and you could say that whichever position gets the most votes is the better argument. However, it's hard to separate the quality of this argument from other things that affect the outcome, such as partisanship, each voter's prior opinions about gun ownership, and external factors such as recent high-profile events or a high-profile candidate endorsing one position or the other.
>
> *Human evaluation*
>
> > Human evaluation tends to be extremely biased on emotionally charged topics like guns and mental illness. You'd need to have a large focus group, test for pre-existing biases, and perform statistical corrections based on these biases.
>
> *Automated metrics*
>
> > It's unclear how BLEU or ROUGE scores, which measure n-grams, correlate with persuasiveness.

Because elections on ballot initiatives involve different questions each time and only two arguments (one for and one against) appear in the voter pamphlet, using the ML and fine-tuning approaches will require a lot of statistical rigor. So, let's take the prompting approach.

The full code for this example is on GitHub—please use the notebook to follow along.

Scoring criteria

Decades of research on voter education materials have resulted in some best practices for designing effective materials and doing political persuasion that you can roll into a set of criteria:

> - Centers the voter: easy for a voter reading this to understand how the initiative will affect them. Consider voters at different socioeconomic statuses and education levels.

> - Organizes information as a pyramid: the most essential information is presented first and inessential details and explanations last.
>
> - Understandable: uses plain language, simple sentences, and minimizes jargon.
>
> - Clarity: the call to action is clear. Why should the voter vote for/against this initiative?
>
> - Caters to undecided: provide additional information that may sway undecided voters for/against the cause's direction. This might be endorsements from authoritative/neutral groups or a specific comparison to the competition.

These criteria can be incorporated into the prompt for LLM-as-judge:

> You will be given an argument for or against a Washington State initiative which will appear in the voter pamphlet that is mailed to all households.
>
> Provide a score in the range 1-5 for each of the following criteria:
>
> {scoring_criteria}
>
> For each score, provide a brief justification.
>
> **Argument**:
>
> {argument}

Using this prompt with OpenAI's GPT-4o-mini on the argument for the initiative gave us the results shown (in part) here:

> 3. **Understandable: 4**
>
> - The language is mostly plain, and the sentences are straightforward, which makes the argument accessible. However, some readers may still find terms like "civil protection orders" somewhat legalistic. Avoiding such phrases or providing brief definitions could improve overall comprehension.

The full code and response are available on GitHub.

Considerations

The LLM-as-Judge pattern suffers from one key drawback (inconsistency) and several biases. There are some situations where caution is warranted.

Inconsistency

LLM scores are nondeterministic because LLM responses can vary. You can reduce inconsistency between scores by caching, setting the temperature to zero, using the same random seed, and so on. However, nailing down the generation is not enough because you do want similar outputs to receive similar scores.

Consider also using these three approaches to make LLM-as-Judge scores more consistent:

Coarse scores

> Avoid creating very fine-grained scores—a score range of 1 to 5 is usually OK, but the inconsistency problem becomes more acute if you set the range to 1 to 10 or 1 to 100. Taking this to the logical extreme, you'll get the best results from LLM-as-Judge if you can pose the score as a binary (yes or no) question.

Multiple criteria

> To reduce the inconsistency problem, consider setting up multiple scoring criteria instead of asking for a single aggregate score—this is a form of CoT (Pattern 13). You can improve on the basic prompt by supplying a few examples (with few-shot learning) to help calibrate the model on each of the criteria. In some situations, providing a single reference result for comparison purposes will calibrate the model and keep the output scores consistent across invocations.

Multiple evaluations

> Instead of performing the evaluation only once, you could have multiple LLMs evaluate the responses. This can be useful if each LLM takes on the role of a different stakeholder and assesses how well the LLM's response aligns with the needs of that stakeholder. This is sometimes termed *LLM-as-jury*.

Of course, you can combine these approaches. For example, one way to get nuanced scoring while posing binary questions is to combine it with LLM-as-jury. This is termed *polling*.

Leniency

LLMs tend to be extremely lenient with scores—like professors who give every student A's and B's. This, combined with LLMs' nondeterministic nature, means that you should avoid comparing the output scores for two pieces of content to determine a ranking. Instead, it's usually better to have your LLM-as-Judge compare the two pieces directly. This is what we did in Pattern 5, Content Optimization (in Chapter 2), for example.

Another way to address this leniency problem is to calibrate responses by using group rewards—this is what DeepSeek does with group relative policy optimization (GRPO), in which multiple responses are generated and scored in tandem, and each response is assigned a score that is normalized by the average score of the group.

A third way to deal with leniency is to lower your expectations on what you use LLM-as-Judge for. For example, you might use the scores only to identify problems—so you can use LLM-as-Judge to assess technical issues in RAG such as context loss, which scores 0 as opposed to the usual 0.95.

Bias

Frontier models tend to perform quite well with the LLM-as-Judge pattern, and there's often no need to look further. However, LLMs tend to offer overly favorable scores when evaluating their own responses. This is called *self-bias*, and because of this, it's usually better to use a different LLM for evaluation than the LLM that was used to produce the content. The self-bias of LLMs is an instance of LLMs preferring content that looks like their training data. Because of this, they tend to prefer well-written text (specifically, text with lower perplexity)—even if it is inaccurate—over badly written but accurate text.

LLMs may also favor lengthy reviews over concise ones, even if both reviews contain the same relevant details. LLMs may miss information that's located in the middle of lengthy text, and they tend to favor information at the beginning or end of an answer. These biases are often referred to as *length bias* and *positional bias* respectively.

Because of the inconsistency and bias issues with using frontier LLMs, it may be worth considering fine-tuned small language models (like PandaLM) that have lower costs. You can use models like PatronusAI for specific evaluation tasks like multimodal responses, and you can also use them for industry-specific custom metrics.

Caveats

In our zero-shot prompt, we asked the LLM to provide an explanation of the score:

> For each score, provide a brief justification

There is some evidence that doing this, while offering interpretability, generally has a negative impact on the model's evaluation performance and bias mitigation. It may be that the process of self-explanation introduces deeper biases. If you don't need explanations, then you can take the probability-weighted mean score of the generated distribution (instead of choosing the most likely score, as would normally be the case when temperature = 0). Just be careful, as this interacts in somewhat weird ways with a CoT approach.[2]

In terms of evaluation, measuring outcomes is the best approach, but it can be hard to tease apart all the contributing factors or *confounding variables*. In such situations, judging based on an easily understandable set of key performance indicators (KPIs) that are aligned with business goals is often the next-best approach, although you do have to watch out for adversarial actors trying to game the metrics. Therefore, when using LLM-as-Judge, try to make the scoring criteria a proxy for either the outcome or the KPIs.

References

In 2022 and 2023, the research and practitioner community organically realized that frontier LLMs had gotten good enough at following instructions that they could be used to evaluate content on arbitrary scoring rubrics. The first academic mention we found was Bai et al. (2023), who called this pattern *Language-Model-as-an-Examiner*. Shankar et al. (2024) discussed the need for alignment and consistency, and Balog, Metzler, and Qin (2025) discussed the many biases and caveats associated with LLM-as-Judge. Prominent practitioners Hamel Husain (2024) and Eugene Yan (2025) also discussed many practical considerations. Gu et al. (2025) carried out a comprehensive survey on the LLM-as-judge technique, variations on it, and challenges associated with it.

DoorDash implements LLM-as-Judge for LLM-based Dasher support automation, and AWS uses it for model evaluation on Amazon Bedrock. Acrolinx uses AI guardrails that it implements with LLM-as-Judge to maintain brand voice consistency across content.

Pattern 18: Reflection

The Reflection pattern is an agentic approach in which an AI system evaluates its own reasoning, decisions, and outputs before finalizing them. This self-monitoring process allows the system to identify errors, improve solutions, and refine its approach to problems.

Here, the AI system *as a whole* is reflecting and improving, and the foundational model used to generate the response remains the same. You'll sometimes see this process referred to as *self-reflection*, but there's no reason why the evaluation has to be done by an LLM at all, let alone by the same LLM that generates the content.

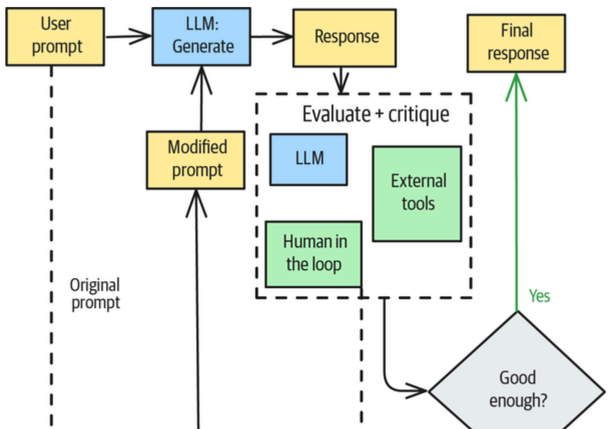
Problem

Suppose you are using an LLM-based tool, such as ChatGPT or Perplexity, through a web interface. If the LLM produces a suboptimal or incorrect response, you can submit a follow-up question, state what's wrong with the response or suggest specific changes, and have the LLM correct its earlier response.

When you invoke an LLM through its API, though, the calls are stateless. So how do you get the LLM to correct an earlier response in response to feedback or criticism, and how do you automatically generate a critique that can be used to correct an earlier response from that LLM?

Solution

Instead of making a single call to the LLM, you invoke the LLM twice or more. You do it the first time with the user prompt, but then, instead of sending the response directly back to the client, you send it to an evaluator (see Figure 6-2). This evaluator could be an LLM (as in Pattern 17, LLM-as-Judge), an external tool, or even a human. The evaluator doesn't just provide a score; it provides a critique explaining how the response falls short. You then use that criticism to apply feedback and create a modified prompt. Finally, you send this modified prompt to the LLM to generate a new response, which is again evaluated. This cycle repeats until the response meets a quality bar.



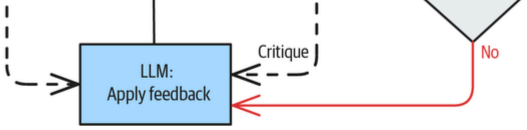


Figure 6-2. Reflection adds an intermediate critiquing step to the typical LLM usage pattern

There are a couple of minor variations.

Maximum attempts

To avoid infinite loops, you can specify a maximum number of retries. The special case of having exactly one retry is quite common because it lets you avoid having to determine what “good enough” means. If you are retrying just once, you could use a zero-shot CoT (Pattern 13) approach and ask it to reconsider the original answer.

Conversational state

In conversational systems, you can treat the evaluator as one of the participants, and the criticism will be a message from that participant. You can then modify the prompt by simply adding the message history (either as is or in summary form) to the previous prompt. For example, you can do this in Autogen:

```
# Create a list of LLM messages to send to the model.
messages: List[LLMMessage] = [*self._system_messages]
for m in self._session_memory[message.session_id]:
    if isinstance(m, CodeReviewResult):
       messages.append(UserMessage(content=m.review, source="Re
    elif isinstance(m, CodeReviewTask):
       messages.append(AssistantMessage(
                            content=m.code_writing_scratchpad,
                             source="Coder"))
    elif isinstance(m, CodeWritingTask):
       messages.append(UserMessage(content=m.task, source="User
    else:
       raise ValueError(f"Unexpected message type: {m}")
```

Example

Let’s look at an end-to-end example of using Reflection to design a company logo. The full code is on GitHub.

Evaluate and critique

The first step is to build a good evaluator. You can use Pattern 17, LLM-as-Judge, and provide a scoring rubric for logos:

> Analyze the following proposed logo for {company}.
>
> {company_description}
>
> Score the logo 1-5 on each of the following criteria:
>
> - It is clear from the logo what the name of the company is.
> - The logo and image are appropriate for what the company does.
> - The logo does not conflict with any well-known brand or competitors.
> - The logo design is streamlined and clean.
> - The logo stands out and is easy to recognize.
>
> Explain your scores.

Try it by sending the preceding prompt to an LLM (we used Claude Sonnet 3.7) along with the logo of the company Pydantic (see Figure 6-3), using this code:

```
agent = Agent(CLAUDE,
              model_settings = {
                      "temperature": 0 # for LLM-as-Judge
              })
result = agent.run_sync([prompt, ImageUrl(url="...")
```

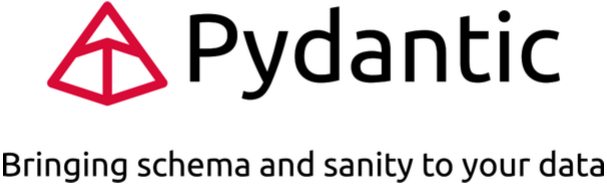

Pydantic
Bringing schema and sanity to your data

Figure 6-3. The logo of the Pydantic company, which we use to illustrate the logo evaluator

You'll get a response that includes criticism similar to the following:

> Appropriateness (4/5): The geometric pyramid/triangle symbol in pink works well for a data validation library. The shape suggests structure, validation, and frameworks - all relevant to Pydantic's core business. The tagline directly addresses data validation. However, it doesn't specifically reference their AI agent framework or Logfire observability platform, which is why I didn't give a perfect score.

Logo designer

You can use a zero-shot prompt on a multimodal zero-shot LLM (Gemini 2.0, in our case) to design a logo, if you provide some basic information about the company:

> Generate a logo image for Hiroshi's Sushi. Hiroshi's Sushi makes delicious Japanese food, and our omakase is the best in the city.
>
> Follow modern design practices.

Figure 6-4 shows the result.

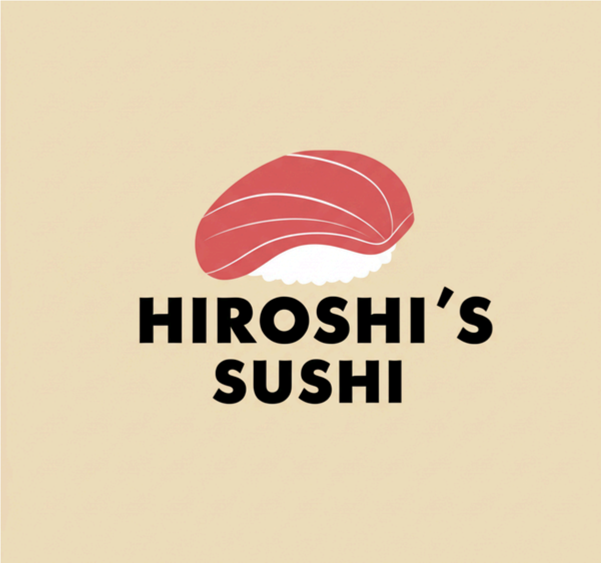


Figure 6-4. Initial logo designed by Gemini for a hypothetical restaurant

After applying the evaluator to the generated image, we got this, in part:

> No conflicts (4/5): The design appears to be unique and doesn't immediately evoke any major sushi chain logos. The simple nigiri illustration is somewhat generic to the industry, but the overall composition with the bold typography creates a distinctive look. I'm giving it a 4 instead of 5 only because there are many sushi restaurants that use similar nigiri imagery, though this particular execution seems unique.

You could calculate the overall score and threshold it to determine whether or not to move forward. However, it's hard to determine an appropriate threshold. For our first attempt at a score, we calculated a weighted average:

```
def score(self) -> float:
        return (10 * self.clarity +
                10 * self.appropriateness +
                30 * self.no_conflicts +
                30 * self.clean_design +
                20 * self.easy_to_recognize) / 500.0
```

The PydanticAI logo received a score of 0.9, but so did the sushi logo. This is, surprisingly, a common problem—since LLMs tend to be lenient scorers, it's hard to come up with a rubric that can distinguish between a "good enough" design and a great one.

A simple workaround is to have exactly one round of criticism. Then, no threshold is required.

Apply Criticism

The next step is to apply the feedback to create an improved prompt for the next iteration. You can use the following prompt to have Claude generate instructions for the logo designer:

> Use the following feedback to generate detailed extra instructions to send back to the designer of the logo.
>
> {critique}

When we did so, the detailed instructions included the following:

> ## Suggested Refinements:
>
> 1. **Increase Distinctiveness (Addressing the 4/5 on Uniqueness)**:
>
>> - Consider adding a subtle, unique twist to the nigiri illustration to differentiate it from other sushi restaurants (perhaps a signature knife mark, a unique shape to the rice, or *a distinctive way the fish drapes over the rice)*
>>
>> - Explore incorporating a small, simple element that represents Hiroshi's specific approach or specialty

Note the impact of the italicized instruction on the right side of Figure 6-5.

Regenerate

Finally, send the previously generated logo, along with the changes to make, to the logo designer:

```
def design_logo(company,
                company_description,
                output_filename,
                previous_logo,
                changes_to_make) -> str:
    client = genai.Client(api_key=os.environ['GEMINI_API_KEY'])

    prompt = f"""
        Here's a proposed logo image for {company}.
        {company_description}

        Please edit the image and make the following changes.
        Return only the final image after all edits.
        {changes_to_make}
    """
    previous_image = Image.open(previous_logo)
    contents = [prompt, previous_image]

    response = client.models.generate_content(
        model=GEMINI,
        contents=contents,
        config=types.GenerateContentConfig(
          response_modalities=['TEXT', 'IMAGE']
        )
    )
```

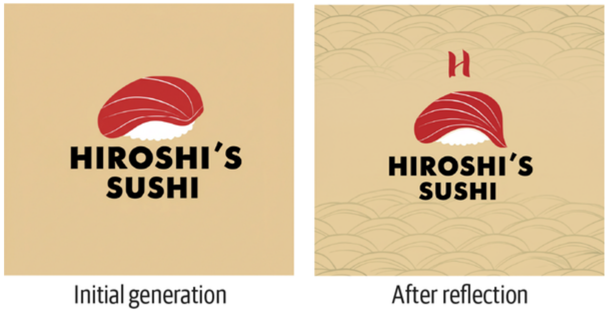
You'd call this function to generate the second and later iterations.

Next, we asked it to generate several drafts:

```
def regenerate(n=3):
    for x in range(n):
        outfile = f"output_logo_2_{x+1}.png"
        design_logo(company, company_description, outfile,
                    previous_logo=image_file,
```

```
                        changes_to_make=extra_instruction)
```

When we did so, we got the logo shown on the right in Figure 6-5 as one of the drafts. It incorporates some of the changes suggested by the evaluator.



Initial generation

After reflection

Figure 6-5. The initial logo designed by Gemini for a hypothetical restaurant (on the left) and the logo after one round of Reflection, incorporating suggestions generated by Claude (on the right)

Considerations

Reflection is one of the four core agentic patterns that was identified by Andrew Ng (see the sidebar in Chapter 5). Reflection has become increasingly crucial in modern AI systems for several reasons, including the following:

Quality and robustness

By reviewing its own outputs, an AI can catch errors before it presents results to users. Logging these reviews also helps identify edge cases and potential failure modes.

Mitigation of reasoning and planning limitations

Reflection allows for iterative refinement of reasoning processes. For novel and open-ended tasks, it's often easier to perform a task once and then determine how to improve it than to plan out the approach beforehand.

Transparency

The reflection process creates explicit reasoning traces that improve explainability.

As AI systems take on more complex and consequential tasks, their ability to reflect has become essential to their reliable performance.

Tradeoff: Cost versus quality

Reflection involves making multiple LLM inference calls, which adds computational costs and latency. You'll need to evaluate the potential improvement in response quality against these costs before you can decide whether to add more rounds of reflection or increase the evaluation threshold.

At the time of writing, many frontier model API providers are struggling with availability. You often need to have a retry strategy just to get the first call to succeed, and Reflection, by introducing the need to make multiple calls to accomplish a task, can increase tail latency beyond acceptable limits.

The cost-versus-quality consideration typically affects different use cases differently. In code generation, using reflection to evaluate the output before submitting the code can reduce syntax errors and logical bugs. The relative cost of doing one more round of reflection compared to the cost of a broken build is often significant enough that such systems involve multiple stages of reflection. They may even incorporate external tools, such as compilers and sandboxes, in the evaluation step. However, reflection may add unacceptable latency in applications such as chatbots and real-time game engines. Therefore, you might find it helpful to develop heuristics for your own business that you can use to adjust the reflection depth, based on the problem characteristics, the available time, and the business impact of the expected improvement in quality.

As in the logo creation example, you can create multiple drafts, but that doesn't have to wait until the final stage. You could generate multiple drafts even in the first step and then follow a beam search–like approach of editing all of them and pruning out poor candidates after each step, before making the final choice. Of course, this would increase costs even more.

Getting evaluation right

The most important part of the Reflection pattern is the evaluation stage, so make sure that your evaluation is robust and identifies what makes a response fit for purpose.

LLMs tend to rate their own work highly, but you can avoid such self-bias by using a different LLM for evaluation than for the actual work. In our example, we used Gemini to create the logo images and Claude to generate the critique. See the “Considerations” section of “Pattern 17: LLM-as-Judge” for other recommendations on how to work around the leniency problem.

References

You can trace the development of Reflection in AI from the idea of cognitive monitoring to improve metacognition (Flavell, 1979) in psychology through reinforcement learning of a theory of mind (Rabinowitz et al., 2018). Verbal feedback, as in Reflexion (Shinn et al., 2023), and self-refinement (Madaan et al., 2023) are used to improve quality and factuality in RAG systems, such as Self-RAG (Asai et al., 2023). They’re also used to reduce the number of bugs in generated code (Dou et al., 2024).

Amazon uses Reflection to ensure that generated product listings are complete and correct. The validation LLM also looks for features that negatively impact listing performance.

Pattern 19: Dependency Injection

The Dependency Injection pattern makes it easier to independently develop and test each component of an LLM chain. When you build chains of LLM calls, build them in a way that makes it easy to inject a mock implementation to replace any step of the chain.

Problem

It can be difficult to develop and test GenAI applications for several reasons:

They are nondeterministic

> Unlike with most software APIs and statistical models, calls to an LLM with the exact same input can return different outputs each time. This makes it difficult to verify that the code is correct by supplying synthetic inputs and checking them against the expected output.

Models change quickly

> You’re often building on foundational models whose upgrade schedule you don’t know. Sticking to a foundational model as of a certain date can help nail down the version, but you lose out on the benefits of the improvements in foundational-model capabilities. The longer and more detailed your prompts, the more brittle they are to changes in the underlying technology.

Your code needs to be LLM agnostic

> If you’re building models that will be used in more than one environment (such as on premises and on Google or on AWS and on Azure), you’ll often want your prompts and code to work on multiple foundational models. While the fact that other vendors support OpenAI’s API and the availability of frameworks like PydanticAI and LangChain make your code portable, they don’t help make your *prompts* portable. Thus, you need to test how your code works on multiple LLMs.

Developing and testing agentic applications that involve chains of LLM calls is particularly difficult—when the output of one model will be embedded into the input context of the next, how can you develop and test each unit of the chain independently of the others?

The solution will make more sense if we discuss an example first.

Example

Let’s say that you’re using an LLM to improve the marketing descriptions of O’Reilly books—meaning the descriptions that appear in online bookstores and on the back covers of physical copies. You’ll implement this in two steps. In the first step, you'll generate several ideas to improve the marketing description, and in the second step, you'll choose the best idea and use it to make the actual change. This is, of course, an example of combining Pattern 13, CoT (from Chapter 5) with Pattern 18, Reflection. The full code is on GitHub.

Step 1: Generate a critique

In the first step, you send the text of the marketing description and get back a critique. The critique is defined as a structured object (see Pattern 2, Grammar, in Chapter 2):

```
@dataclass
class Critique:
    target_audience: List[str]
    improvements: List[str]
```

Next, you call the LLM with an appropriate prompt:

```
def critique(in_text: str) -> Critique:
    prompt = f"""
    You are an expert marketer for technology books.
    You will be given the marketing description for a book.
    Identify the target audience by roles (eg: Data Analyst, Da
    Suggest exactly 5 ways that the *marketing description* can
    that it appeals better to this target audience.
    Do not suggest improvements to the book itself.

    **Marketing Description**:
    """
    agent = Agent(GEMINI,
                  result_type=Critique)
    print(f"Invoking LLM to critique text")
    result = agent.run_sync([prompt,
                             in_text])
    return (result.data)
```

Try this out on the marketing description of the book *Machine Learning Design Patterns* (O'Reilly), by your favorite authors. You'll get something like this:

> **Target audience**:
>
> Data Scientist,Machine Learning Engineer,AI Researcher
>
> **Suggested changes**:
>
> Use more specific job titles (e.g., Machine Learning Engineer, AI Researcher) instead of the general term "data scientists."
>
> Highlight the practical applications of the design patterns and how they can directly improve the efficiency and effectiveness of their work.
>
> Emphasize the scalability and maintainability aspects of the solutions, as these are critical concerns for professionals deploying ML systems in production.
>
> Include a section on how the book helps in troubleshooting and debugging common ML issues, which is a frequent pain point for practitioners.
>
> Add testimonials or endorsements from well-known figures in the machine learning community to build credibility and trust with the target audience.

This looks right, but is it right?[3]

Testing the first step

How can you make sure that the preceding code will continue to work well and that it is correct? You do not want to rely on "eyeballing" the result (which is often derogatorily referred to as *vibe checking*). Human feedback—both explicit (such as giving a thumbs-up or thumbs-down to the suggestions) and implicit (such as presenting the preceding list of suggestions to a user and seeing which one they select)—has its place in product design, but you still want your test suite to test the correctness of the code.

To test the first step, you can perform assertions on the returned response to make sure it meets expectations:

```
def assert_critique(critique: Critique):
    assert len(critique.improvements) > 3, "Should have 4+ impr
    assert len(critique.target_audience) > 0, "Should have 1+ r
```

You can also perform LLM-as-Judge (Pattern 17) here for more nuanced checks on the generated content. For example, the previous fourth suggestion reads as if it were a recommendation to add content to the book itself, not just the back cover:

> Include a section on how the book helps in troubleshooting and debugging common ML issues, which is a frequent pain point for practitioners.

To address this, you'd put the assertion code into a test suite that you invoke routinely, such as before every deployment or every commit to the code base.

Step 2: Implement a suggestion

The second step is to implement one of the suggestions. Here, the result is a change log and an improved marketing description:

```
@dataclass
class Improvement:
    change: str
    reason: str
    modified_marketing_description: str
```

This step also involves an LLM call, but one of the inputs into this function is the `Critique` object that was generated in Step 1:

```
def improve(marketing_text: str, c: Critique) -> Improvement:
    prompt = f"""
    You are a helpful marketing assistant.
    You will be given the marketing description for a book,
    its target audience, and a list of suggested changes.

    Pick one change from the list that best meets these criteri
    - It does not require changing the book itself, only the ma
description.
    - It will make the book much more appealing to the target a
    - It requires only 1-5 lines to be changed in the text of t
description.
    Then, make the change and return a change log and the modif

    **Marketing Description**:
    {marketing_text}

    {c}
    """
    print(f"Invoking LLM to improve text")
    agent = Agent(GEMINI,
                  result_type=Improvement)
    result = agent.run_sync(prompt)
    return (result.data)
```

If you do this with the original marketing description, you'll get something like this:

> **Change**:
>
> Use more specific job titles (e.g., Machine Learning Engineer, AI Researcher) instead of the general term "data scientists."
>
> **Reason**:
>
> The target audience includes Machine Learning Engineers and AI Researchers, so using these specific job titles instead of the general term "data scientists" will make the book more appealing to them. This change requires only one line to be modified in the marketing description and does not require changing the book itself.
>
> **New description**:
>
> The design patterns in this book ... proven methods to help Machine Learning Engineers and AI Researchers tackle common problems ...

Again, this looks right, but is it right?

Testing the second step

You can test the second step to confirm that the chosen change is one of the suggestions from the first step and the resulting changes don't modify more than five lines of text.

You should also add assertions for both of these conditions so that the test fails with informative error messages:

```
def assert_improvement(improvement: Improvement, orig_text: str
    assert improvement.change in c.improvements,
           "Chosen change not in original list"
    nlines_changed = ... # use difflib: see GitHub for code
    assert nlines_changed > 0 and nlines_changed <= 5,
           f"{nlines_changed} lines changed, not 1-5"
```

However, this presents a problem: you have to invoke the LLM in Step 1 in order to get the `Critique` object to pass to the second step—so there seems to be no way to develop and test Step 2 independently of Step 1:

```
def improvement_chain(a_text):
a_critique = critique(a_text)
improved = improve(a_text, a_critique)
assert_improvement(improved, a_text, a_critique)
```

But is there a way to test Step 2 independently of Step 1?

Solution

The solution is to replace Step 1 with a mock implementation that returns a hard-coded result while Step 2 is being developed and tested (see Figure 6-6). *You replace LLM calls and external tools with lightweight mocks during development and testing of other components.*

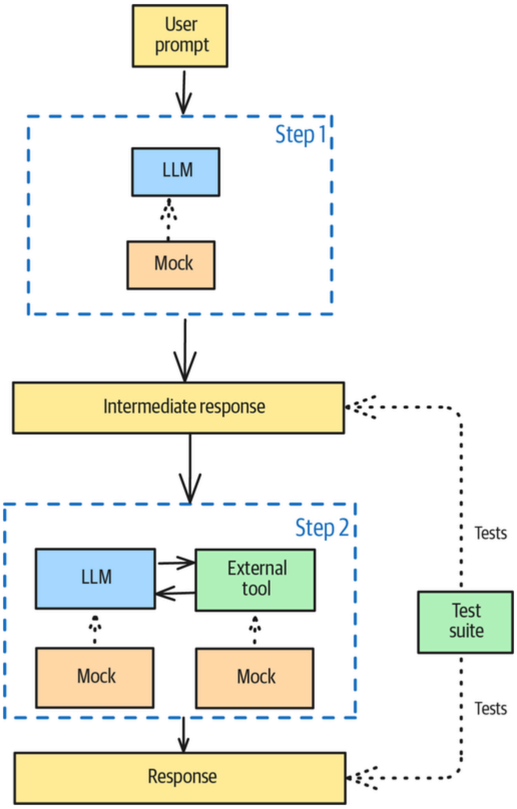


Figure 6-6. Replacing Step 1 with a mock implementation during development of Step 2

Mockable steps

Define the chain of calls in such a way that any one step can be replaced by a different implementation that has the same signature (that is, it will take the same inputs and provide the same outputs). For example, do this:

```
def improvement_chain(
```

```
          in_text: str,
          critique_fn: Callable[[str], Critique] = critique,
          improve_fn: Callable[[str, Critique], Improvement] = impr
                      ) -> Improvement:
    c = critique_fn(in_text)
    assert_critique(c)

    improved = improve_fn(in_text, c)
    assert_improvement(improved, in_text, c)

    return improved
```

The chain *depends* on these functions. You *inject* a different implementation, which is a mock in this case, when you call the chain—that's what gives the pattern its name. The default `critique_fn` is the `critique()` function you wrote, so the default behavior of this chain is to call the LLM for both steps.

In Python, the `assert` statement behaves differently under different conditions. If you run the code with a `-O` flag, it will turn off all assertions. That way, you can have assertions on during development and off in production, without changing the code itself. Running the code in a test environment (Pytest) decorates the `assert` statements and provides detailed information about the call stack where the assertions failed.

Mocking Step 1

When developing Step 2 or to test it independently of Step 1, you create a mock implementation of the `critique()` function that has hardcoded values:

```
def mock_critique(in_text: str) -> Critique:
    print(f"Using mock to critique text")
    return Critique(
        target_audience = """AI Engineers,Machine Learning Engi
Engineers
        """.split(','),
        improvements = """
Use more precise language to define the problems the book solve
Add specific examples of how the design patterns have been used
real-world problems.
Highlight the benefits of using design patterns, such as increa
reduced costs, and improved accuracy.
Emphasize the book's practical approach, with step-by-step inst
examples.
Include testimonials from data scientists who have used the des
the book to improve their work.
        """.strip().split('\n')
    )
```

Now, use this mock in place of the `critique()` function in the chain:

```
improved = improvement_chain(mldp_text, critique_fn=mock_critiq
```

Considerations

We've illustrated mock functions here, but you can mock objects, too, using features such as abstract classes and inheritance. Make sure to do so idiomatically in the language you're using (Go, TypeScript, etc.).

The hardcoded mock values can be difficult to get right as the interaction between the steps increases. In the preceding example, the suggestions are generic enough that the hardcoded instructions work for a variety of books. If this isn't the case, selecting the right mock object to return can itself become a challenge.

In addition to steps that involve LLM calls, you could mock external functions to keep you from being at the mercy of network latency and service availability during development and testing.

References

In software engineering, Dependency Injection is a longstanding pattern (per Fowler, 2024)—enterprise software frameworks such as Spring are built around this concept. However, many AI engineers who come from a science background are unaware of it. At the time of writing, Dependency Injection is not supported natively by any of the GenAI frameworks, although Pydantic comes close with dependency injection of prompts and tools into agents, but not of the agents themselves. We hope that this will change.

Pattern 20: Prompt Optimization

Prompt Optimization provides a systematic way to update the prompts used in a GenAI pipeline whenever its dependencies change. It does this by optimizing the prompts on a dataset of examples.

Problem

The process of building a GenAI application typically involves trial and error. The input into a foundational model is typically a text prompt. To get better results, the first thing you'd try is prompt engineering or changing the prompt—you might make the prompt more detailed, add a few examples, reorder the order of instructions, and so on.

If you change the foundational model (such as when your model provider releases a new version), all your trials need to be repeated. This makes your application as a whole very brittle to changes in the underlying LLM—or changes to any other dependency (such as in the toolchain).

Is there an easy way to update your prompts when your dependencies change, to make sure that you are maintaining the same level of performance?

Solution

There is a famous aphorism called the fundamental theorem of software engineering, and perhaps appropriately, its origins are convoluted. The term was coined by programmer Andrew Koenig to describe a quote that computer science pioneer Butler Lampson attributed to computer science professor and pioneer David J. Wheeler. The theorem states, "All problems in computer science can be solved by another level of indirection."

Maybe not all, but the need to optimize prompts for a given LLM version *is* a problem that an extra level of indirection can help to solve.

The extra level of indirection here is provided by a framework that can take a pipeline of LLM calls and automatically optimize it by injecting multiple variations of the prompts. Prompt Optimization requires four components (also see Figure 6-7):

A pipeline of steps

> You need to set up a pipeline consisting of the key steps in your application. This pipeline will take an input, invoke the foundational model (and tools) as needed, and return an output. The framework will inject the necessary prompts for the steps.

A dataset

> You need a dataset of examples on which to evaluate prompt iterations. These could be pairs of inputs and reference answers (for supervised training), or they could consist only of inputs. Your dataset could be as small as one example (in which you'll get the best prompt for that one input), or it could consist of a handful of examples.

An evaluator

> You need an evaluator that can automatically evaluate the result of running the pipeline on an input. This evaluation could be based on comparing the result of the pipeline to a reference answer (for supervised training) or calculating a fitness score (as in Pattern 17, LLM-as-Judge; this is useful when you don't have reference answers).

An optimizer

> You need an optimization framework that can generate multiple prompt variations, run them against your dataset, and return an optimized pipeline. This optimized pipeline is the input pipeline, in which each step is populated with the prompt that performed best when evaluated on the set of examples.

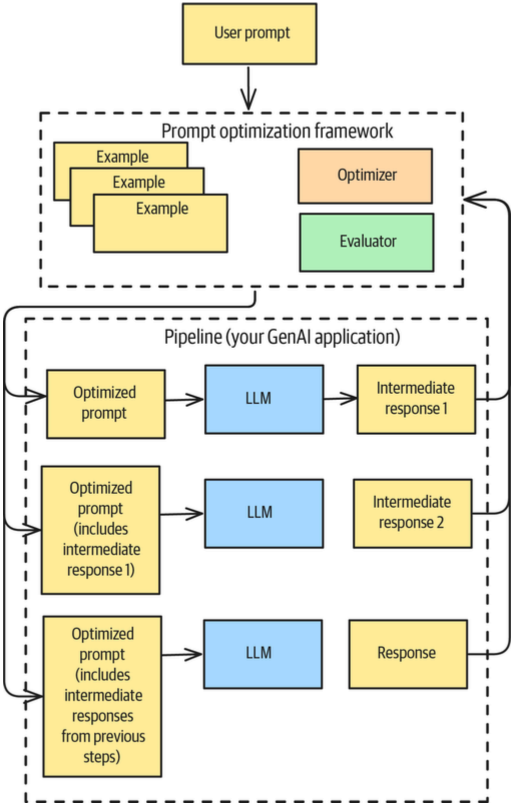

User prompt
Prompt optimization framework
Example
Example
Example
Optimizer
Evaluator
Pipeline (your GenAI application)
Optimized prompt
LLM
Intermediate response 1
Optimized prompt (includes intermediate response 1)
LLM
Intermediate response 2
Optimized prompt (includes intermediate responses from previous steps)
LLM
Response

Figure 6-7. Prompt Optimization involves optimizing the prompts for a pipeline of steps on a dataset of examples

In the next section, you'll see how these components work in DSPy, which is a framework that supports prompt optimization.

Example

Suppose you want to improve the information that appears on the back cover of O'Reilly books. This includes a description of the book, what readers can expect to learn, and *blurbs* (which are quotes from reviewers who have praised the book). Today, those descriptions are based on drafts by the books' authors who are often poor marketers of their own books. The full code for this example is on GitHub, so please follow along with us.

Creating a pipeline

The first step is to configure DSPy with the LLM you want to use. Even though you can use a different LLM for each step of the pipeline, we'll keep it simple and use the same LLM for all the calls:

```
import dspy
lm = dspy.LM("claude-3-7-sonnet-latest",
             api_key=os.environ['ANTHROPIC_API_KEY'])
dspy.configure(lm=lm)
```

As you'd expect, DSPy supports all the major LLMs.

The first step of the pipeline is to extract the various parts from the text. The step will have one input (the text that appears on the back cover) and one output (a Python data class), so you define its signature as follows:

```
class BlurbExtraction(dspy.Signature):
    text: str = dspy.InputField(desc="Text from backcover")
    blurb: Blurb = dspy.OutputField(desc="Extracted informatio
```

For simplicity, we'll call the entire information on the back cover the blurb, although the blurb is technically only praise quotes. Defined this way, the blurb on the back of an O'Reilly book has four parts, so you can define the output `Blurb` class as follows:

```
class Blurb:
    about_topic: str = Field(description="""Why the topic of b
```

```
learning""")
    about_book: str = Field(description="What book contains")
    target_audience: List[str] = Field(description="""Roles suc
Engineer, Data Analyst that the book is for""")
    learning_objectives: List[str] = Field(description="""4-6 l
that complete the sentence: You will learn how to ___""")
```

To provide more information about the variables than you can fit into the variable name, you can initialize them with description strings, as we did previously.

To run the pipeline, you can ask DSPy to automatically create a CoT module with the given signature:

```
mldp_text = "The design patterns in ... fairly" # text from bac
module = dspy.ChainOfThought(BlurbExtraction)
orig_cover_info = module(text=mldp_text)
```

This code will extract the cover information from the blurb.

The next step in the pipeline is to improve the blurb. As with `BlurbExtraction`, you need to define the signature for `BlurbImprovement,` in which the description of each field captures what the LLM needs to do:

```
class BlurbImprovement(dspy.Signature):
    current_cover: Blurb = dspy.InputField(desc="Current inform
    about_topic: str = dspy.OutputField(desc="""More catchy sta
is worth learning""")
    about_book: str = dspy.OutputField(desc="""More appealing (
audience) description of book contents""")
    target_audience: List[str] = dspy.OutputField(desc="""more
of roles. Instead of programmer, say software engineer. Restric
    learning_objectives: List[str] = dspy.OutputField(desc="""I
rephrased or reordered to be more appealing to target audience.
```

Now, combine the two steps into a single module so that you can optimize both steps together:

```
class BlurbPipeline(dspy.Module):
    def __init__(self):
        self.extract_info = dspy.ChainOfThought(BlurbExtraction
        self.improve_blurb = dspy.ChainOfThought(BlurbImproveme

    def forward(self, in_text: str) -> (Blurb, Blurb):
        cover_info = self.extract_info(text=in_text)
        improved_cover = self.improve_blurb(
                                       current_cover=cover_info
        return cover_info.blurb, make_blurb(improved_cover.toDi
```

The `forward()` method in the module calls the steps one after the other, passing the output of the first step as the input of the second.

To run the pipeline, use the following code:

```
program = BlurbPipeline()
orig_blurb, improved_blurb = program(in_text=...)
```

How well does it do?

Results

The current text on the back cover of the *Machine Learning Design Patterns* book identifies the following learning objectives:

> You'll learn how to:
>
> * Identify and mitigate common challenges when training, evaluating, and deploying ML models
>
> * Represent data for different ML model types, including embeddings, feature crosses, and more
>
> * Choose the right model type for specific problems
>
> * Build a robust training loop that uses checkpoints, distribution strategy,

> and hyperparameter tuning
>
> * Deploy scalable ML systems that you can retrain and update to reflect new data
>
> * Interpret model predictions for stakeholders and ensure models are treating users fairly

You asked the LLM to rephrase and reorder these learning objectives to make them more appealing to the target audience. Here's that part of the result:

> Machine Learning Architects, AI Systems Engineers, and Enterprise Data Scientists will learn how to:
>
> * Architect resilient ML systems that overcome common training, evaluation, and deployment challenges
>
> * Engineer sophisticated data representations, including embeddings and feature crosses that unlock model performance
>
> * Select optimal model architectures tailored to your specific business problems and constraints
>
> * Implement production-grade training pipelines with checkpointing, distributed training, and automated hyperparameter optimization
>
> * Design scalable ML systems with seamless retraining capabilities and efficient data pipeline integration
>
> * Deliver transparent, explainable, and ethically sound ML solutions that build stakeholder trust

It seems to have done that quite effectively. Can we measure the quality of this changed description?

Evaluating the blurb

To create an evaluator that measures how good a candidate blurb is, you can use LLM-as-Judge. While you can create a rubric for each component of the blurb, you know from the "Considerations" section of "LLM-as-Judge: Pattern 17" that it tends to be quite lenient. If one of the blurbs is rated 0.89 and the other is rated 0.90,[4] it's difficult to trust in the improvement. You'll get better results if you give the LLM the original text as a reference blurb and ask it to compare the rewritten blurb to the original:

```
class BlurbScore(dspy.Signature):
    reference_blurb: Blurb = dspy.InputField()
    blurb_to_evaluate: Blurb = dspy.InputField()
    topic_score: float = dspy.OutputField(desc="""-1 to 1: how
appealing the topic description is, as compared to reference"""
    contents_score: float = dspy.OutputField(desc="""-1 to 1: h
appealing (to target audience) the book content description is,
reference""")
    objectives_score: List[float] = dspy.OutputField(desc="""-1
how appealing each learning objective is to target audience, as
reference""")
```

To run the scorer module, pass in the output of the improvement pipeline:

```
scorer = dspy.ChainOfThought(BlurbScore)
score_pred = scorer(orig_blurb, improved_blurb)
```

Then, aggregate the scores, penalizing blurbs that are too long:

```
def calc_aggregate_score(blurb: Blurb, p: dspy.Prediction) -> f
    result = (p.topic_score * 10 +
              p.contents_score * 10 +
              sum(p.objectives_score)) /
              (20 + len(p.objectives_score))
    ## cut 0.1 for every line beyond MAX_LINES
    marketing_copy = blurb.toMarketingCopy()
    num_lines = len(marketing_copy.splitlines())
    if num_lines > TARGET_MAX_LINES:
        result -= 0.1 * (num_lines - TARGET_MAX_LINES)
    if result < 0:
        result = 0
```

```
    return result
```

The fitness score for the rewritten one is 0.63, which indicates a considerable improvement over the original.

The generated prompts

All you've done here is to specify the input and desired output descriptions for each step. The actual prompts to the language model were generated by DSPy.

This is the prompt that was generated when we performed the blurb improvement step, for example:

> Your input fields are:
>
> 1. `current_cover` (Blurb): Current information on book
>
> Your output fields are:
>
> 1. `reasoning` (str)
>
> 2. `about_topic` (str): More catchy statement why topic is worth learning
>
> ...

These are just the input and output variables specified in the module signature (see the full code on GitHub), except that a reasoning field has been added to the set of outputs in order to implement CoT to generate the reasoning part of the preceding template.

For the book used as the input, the generated reasoning is as follows:

> The current cover information is solid but could be more compelling. I'll enhance it by:
>
> 1. Making the "about topic" section more urgent and exciting by emphasizing how design patterns give practitioners a competitive edge
>
> 2. Making the "about book" section more dynamic by highlighting the practical, transformative nature of the content
>
> 3. Elevating the target audience descriptions to more aspirational titles
>
> 4. Reframing the learning objectives to emphasize career advancement and practical outcomes rather than just skills acquisition

Can we do better? As we mentioned, DSPy is a prompt optimization framework. It supports quite a few optimizers, so let's look at a couple.

Optimizing on one blurb

The simplest thing to do is to have DSPy generate multiple variants of the prompts and see which one produces the best output. If the only blurb you want to improve is the one that appears on the back cover of *Machine Learning Design Patterns*, you can ask DSPy to generate 10 variants of the prompts for the pipeline and give you the best outcome:

```python
def score_reward(args, pred) -> float:
    orig_blurb, improved_blurb = pred

    scorer = ScorerPipeline()
    return scorer(orig_blurb, improved_blurb)

optimized_pipeline = dspy.BestOfN(
    module=BlurbPipeline(),
    N=10,
    reward_fn=score_reward,
    threshold=0.95
)
```

The preceding code specifies a `reward` function that tells the optimizer how good the output, `pred`, of each iteration is. The optimization stops at 10 iterations (`N=10`) or when it reaches a threshold performance measure (0.95).

When we did this, we got a score of 0.74—which is higher than the 0.63 we got earlier. This is the start of that blurb:

> Machine learning design patterns are the secret weapons of elite AI teams, distilling decades of hard-won expertise into actionable blueprints for success. In an era where ML systems increasingly drive critical business decisions, these patterns provide the architectural foundation needed to build robust, scalable, and ethical AI solutions that avoid costly pitfalls that plague most projects.

As an author of the book and the one who drafted the original description, “elite AI teams” makes me cringe. Good thing the LLM has no such qualms—I can see how this would be much more appealing to potential buyers.

You carried out *N* trials and chose the best one, and the variations among the trials were in how the reasoning string in CoT was constructed. The problem with this approach is that you have to run inference 10 times. Is there a way to create a better prompt that would work for any book and run inference only once?

The few-shot optimizer

In addition to CoT, you could add a few examples to the prompt (via few-shot learning; see Chapter 1). The idea is that, if you have 10 examples, you’ll pick 3 of them to serve as examples and then evaluate how well this prompt works on the remaining 7. You bootstrap by selecting a different set of 3 examples each time and picking the prompt that works best.

To do this, first set up a dataset of a few examples:

```
with open("blurbs.txt") as ifp:
    blurbs = ifp.read()
    ...

blurbs = [
    dspy.Example(in_text=b).with_inputs("in_text")
    for b in blurbs]
```

Then, run the optimizer on the dataset:

```
optimizer = BootstrapFewShot(metric=evaluate_blurb)
optimized_pipeline = optimizer.compile(BlurbPipeline(), trainse
optimized_pipeline.save("optimized_pipeline", save_program=True
```

The `evaluate_metric` function is similar to the `score_reward` in the previous section. The result is a pipeline into which optimized prompts have been injected. This optimized pipeline would work well if you wanted to improve descriptions for many books.

You can then save the optimized pipeline and use it for inference:

```
orig_blurb, optimized_blurb = optimized_pipeline(
    in_text=mldp_text
)
```

More to the point, every time the LLM version changes, you can rerun the pipeline, get a prompt that performs well across many books, and continue on. Your code itself doesn’t have prompts.

Considerations

A naive solution to make it easier to update your application is to use a *prompt library*. If you externalize all your prompts to a configuration file, it’ll be easier to change the prompts over time. However, this sort of *prompt management* doesn’t solve the core problem, which is that you’ll still have to experiment manually whenever your dependencies change. Prompt management and versioning are good practices, though. We intentionally didn’t list them as patterns because Prompt Optimization provides a more encompassing solution.

You can extend this prompt management approach to save all the different prompt iterations you try and then have the framework repeat the steps on the next version of the LLM. You can then examine the results (or use an automatic evaluator) and choose the prompt that works best. However, the problem is that as your application changes, your old prompts will also need to be updated—to add or remove variables, for example. Prompt Optimization is much less cumbersome to maintain.

We showed you how the Prompt Optimization pattern can use either multiple trials or bootstrapping to systematically update the prompts whenever your dependencies change. You're not limited to best-of-N or few-shot learning—if your dataset contains thousands of examples, DSPy allows you to fine-tune the LLM for this specific pipeline.

At the time of writing, other frameworks (besides DSPy) that support Prompt Optimization are AdalFlow and PromptWizard. PydanticAI is considering supporting it. We hope that, by the time you read this, Prompt Optimization will be supported by a broader set of frameworks.

Prompt Optimization is often the foundational step you need to take in order to improve the quality of LLM responses over time. Once you have the infrastructure to record all prompts and human feedback, better optimization methods open up. You can export the prompts and human feedback to a dataset to build an assessment prompt (LLM-as-Judge) by using Prompt Optimization. LLM-as-Judge gives you a way to scalably evaluate LLM responses, and then, you can use the captured prompts (even the ones without human feedback) to create a dataset for post-training an LLM that is tuned to the specific task.

References

DSPy was introduced by Omar Khattab and his colleagues at Stanford in the paper "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines" in 2023.

Summary

Reliability is a critical concern when building GenAI applications, since the foundational models on which they are built can produce inconsistent, incorrect, or hallucinated outputs that undermine user trust. This chapter presents four patterns that address different aspects of reliability challenges: LLM-as-Judge enables systematic evaluation of outputs through prompting, machine learning, and fine-tuning approaches; Reflection empowers models to critique and improve their own responses; Dependency Injection creates testable, mockable components for robust application architecture; and Prompt Optimization systematically refines prompts to maximize reliability across diverse inputs. Table 6-1 summarizes these patterns.

Table 6-1. Patterns for increasing reliability

Patterns	Problems	Solutions	Usage scenarios
LLM-as-Judge (Pattern 17)	Evaluation of GenAI capabilities is hard because the tasks that GenAI performs are open-ended.	Provide detailed, multi-dimensional feedback that can be used to compare models, track improvements, and guide further development.	Evaluation is core to many of the other patterns and to building AI applications effectively.
Reflection (Pattern 18)	How can you get the LLM to correct an earlier response in response to feedback or criticism?	The feedback is used to modify the prompt that is sent to the LLM a second time.	Reliable performance in most complex tasks where the approach can't be predetermined.
Dependency Injection (Pattern 19)	You need to independently develop and test each component of an LLM chain.	When you build chains of LLM calls, build them so that it's easy to inject a mock implementation to replace any step of the chain.	Any situation in which you chain LLM calls or use external tools.
Prompt Optimization (Pattern 20)	You need to easily update prompts when dependencies change to	Systematically set the prompts used in a GenAI pipeline by optimizing them on a dataset of examples.	Any situation in which you have to reduce the maintenance overhead associated

	maintain level of performance.	with LLM version changes (and other dependencies).

Together, these patterns form a comprehensive toolkit for increasing the reliability of LLM applications across various dimensions. LLM-as-Judge and Reflection focus on output quality assessment, while Dependency Injection addresses structural reliability through better testing practices, and Prompt Optimization ensures consistent performance through systematic prompt engineering. By implementing these patterns appropriately, developers can significantly enhance the trustworthiness of their LLM applications and therefore make them more suitable for production environments where reliability is paramount.

---

[1] *Overfitting* in ML occurs when a model learns even the noise and random fluctuations in the training data, which makes the model poor at generalizing to new, unseen data. Typically, overfitting happens when you have too little training data relative to the number of trainable parameters in the model.

[2] See Wang et al.’s article “Improving LLM-as-a-Judge Inference with the Judgment Distribution” for details.

[3] As with so many things in business, it comes down to who the intended audience is. Is the book description meant for the person searching the internet or the person looking at the book in a physical or online bookstore? The answer to the question about intended audience changes the evaluation criteria. Using more specific titles would likely narrow the number of people the book reaches, whereas a more general term is better for SEO and reach.

[4] This is not a hypothetical—it’s exactly what happened when we evaluated each blurb by itself.

Table of contents collapsed